

RKNN3 SDK 快速入门

文件标识：RK-JC-YF-416

发布版本：V1.1.0

日期：2026-08-03

文件密级：☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文档面向首次使用RKNN3 SDK的用户，目标是在最短路径内完成环境检查、模型转换、板端部署和首次推理。完整的软件栈、模型开发流程、API说明、Server高级能力及故障排查请参考对应的主文档。

读者对象

本文档主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本	修改人	修改日期	修改说明	核定人
V0.1.0	HPC	2025-05-13	初始版本（RK182X）	熊伟
V0.2.0	HPC	2025-08-15	1. 调整模型加载接口说明 2. 新增模型初始化接口说明 3. 调整模型部署流程图 4. 增加准备开发板 5. 增加环境准备 6. 增加运行示例程序 7. 修改开发流程 8. 增加常见问题	熊伟
V0.3.0	HPC	2025-09-10	更新 LLM 会话相关接口改动	熊伟
V0.3.0b0	HPC	2025-09-12	1. 修改文档中的一些错误 2. 增加附图 3. 增加OpenAI接口使用说明	熊伟
V0.4.0b0	HPC	2025-10-18	1. 增加RK1828的平台	熊伟
V0.5.0	HPC	2025-11-29	1. 增加同步输入、输出数据	熊伟
V1.0.0	HPC	2026-01-01	1. 更新rkllm3-server用法 2. 更新支持的模型框架 3. 更新模型评估方法	熊伟
V1.0.4	HPC	2026-05-12	1. 支持RK3572	熊伟
V1.1.0	HPC	2026-08-03	1. 更新rkllm3-server用法 2. 文档结构重构	熊伟

目录

- 1. 开始之前
 - 1.1 本指南目标
 - 1.2 选择部署模式
 - 1.3 选择示例类型
 - 1.4 支持平台和版本要求
 - 1.5 获取SDK及配套仓库
- 2. 硬件连接与检查
 - 2.1 协处理器模式（RK182X）
 - 2.1.1 硬件清单
 - 2.1.2 开发板和连接工具介绍
 - 2.1.3 连接开发板
 - 2.2 非协处理器模式（RK3572）
 - 2.2.1 硬件清单
 - 2.2.2 开发板和连接工具介绍
 - 2.2.3 连接开发板
 - 2.3 检查设备连接状态
- 3. 主机端和板端环境准备
 - 3.1 安装RKNN3 Toolkit
 - 3.1.1 安装Python环境
 - 3.1.2 安装rknn3-toolkit
 - 3.2 检查Toolkit安装结果
 - 3.3 安装或检查板端RKNPU3环境
 - 3.3.1 安装Runtime库和通信服务
 - 3.3.2 检查版本
 - 3.4 安装编译工具
 - 3.4.1 安装CMake
 - 3.4.2 安装编译器
 - 3.4.2.1 确认开发板系统类型和架构
 - 3.4.2.2 Android系统开发板安装NDK
 - 3.4.2.3 Linux系统开发板安装GCC交叉编译器
 - 3.5 运行环境预检查
- 4. 运行第一个通用模型
 - 4.1 获取示例
 - 4.2 转换模型

- 4.3 编译部署程序
 - 4.3.1 Android
 - 4.3.2 Linux
- 4.4 板端运行
- 4.5 验证输出结果
- 5. 运行第一个LLM
 - 5.1 获取示例
 - 5.2 转换模型
 - 5.3 编译部署程序
 - 5.3.1 Android
 - 5.3.2 Linux
 - 5.4 板端运行
 - 5.5 验证输出结果
- 6. 启动RKLLM3 Server
 - 6.1 常用参数
 - 6.2 启动服务
 - 6.3 健康检查
 - 6.4 使用curl发送请求
 - 6.5 使用Python OpenAI库发送请求
 - 6.6 检查响应结果
- 7. 常见启动问题与下一步
 - 7.1 设备无法识别
 - 7.2 设备/协处理器连接问题
 - 7.3 Toolkit安装验证失败
 - 7.4 转换模型时找不到'py_utils'
 - 7.5 GRQ量化失败
 - 7.6 板端Runtime版本不匹配
 - 7.7 后续文档导航

1. 开始之前

本指南以最短路径引导用户完成RKNN3 SDK的首次运行，涵盖环境检查、模型转换、板端部署和首次推理等最小必要步骤。模型优化原理、完整API说明、Server高级功能和系统化故障排查不在本指南范围内，请查阅对应文档。

本指南覆盖部署模式选择、环境预检查、首次通用模型运行、首次LLM运行、RKLLM3 Server启动与请求验证，以及首次运行中的常见问题排查。

说明： 完整的开发流程、模型转换原理、精度/性能/内存分析等，请参考《RKNN3_SDK_开发指南》及对应API参考文档。硬件、操作系统和版本支持范围以SDK发布包中的兼容矩阵为准，本指南不再单独列出。

1.1 本指南目标

完成本指南后，用户应能够：

1. 确认开发主机、开发板和RKNN3运行环境可用；
2. 使用RKNN3 Toolkit完成一个通用模型的转换并在板端运行；
3. 完成一个LLM模型的转换并在板端运行；
4. 启动 `rkllm3-server` 并通过 `curl` 完成一次OpenAI兼容接口请求；
5. 根据文档导航进入完整开发、API参考或故障排查文档。

1.2 选择部署模式

RKNN3 SDK支持协处理器模式和非协处理器模式。两种模式的主要区别如下：

部署模式	硬件组成	运行方式	主控与NPU关系
协处理器模式 (RK182X)	主控SoC + RK1820/RK1828协处理器	主控SoC负责任务调度和整体控制，RK1820/RK1828负责AI推理	通过PCIe/USB/Ethernet进行数据交互
非协处理器模式 (RK3572)	RK3572 SoC	AI推理任务直接在RK3572上运行	NPU集成在SoC内部，与CPU/GPU共享内存空间

两种模式的软件框架如下：

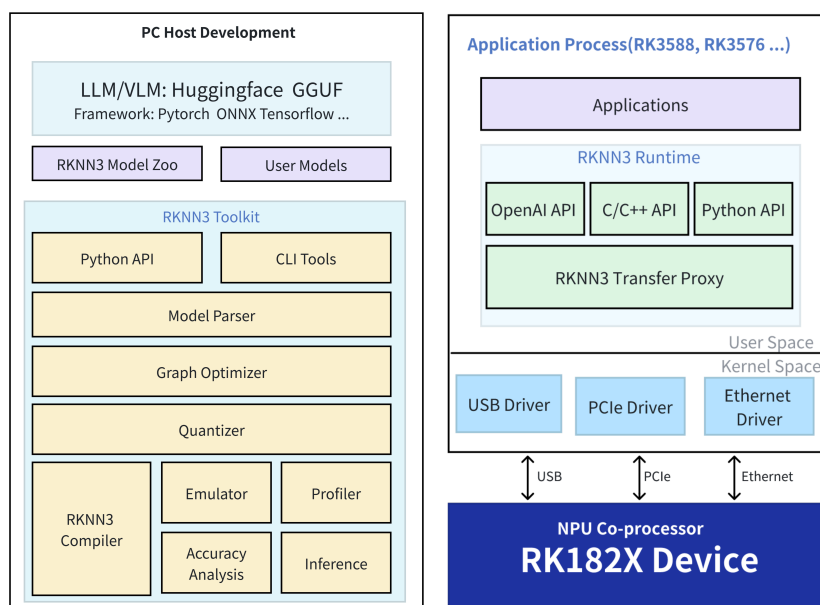


图1-1 RKNN3 SDK框架图（协处理器模式）

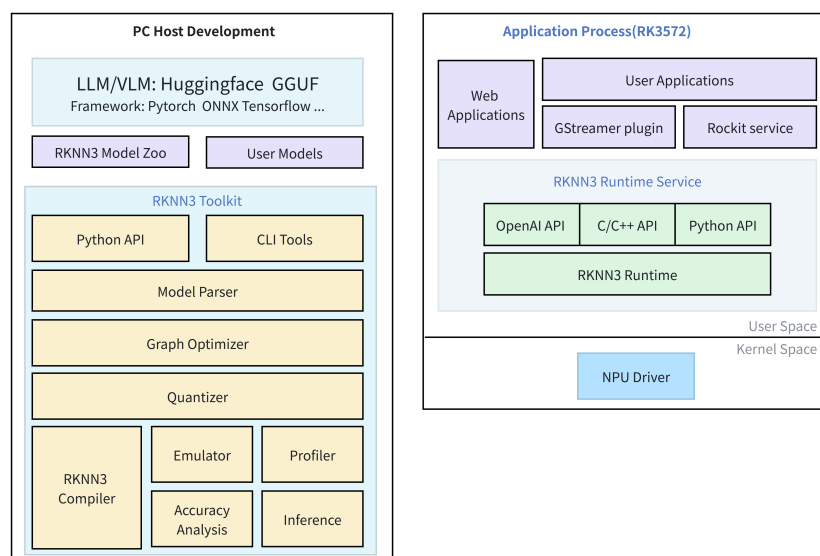


图1-2 RKNN3 SDK框架图（非协处理器模式）

软件框架主要由PC端开发套件和开发板运行环境组成。

PC端开发套件： 用户可通过RKNN3 Toolkit将PyTorch、ONNX等深度学习框架训练的模型转换为RKNN格式。RKNN3 Model Zoo提供模型转换与部署示例，主要覆盖以下模型类型：

- **CNN模型：** MobileNetV2、ResNet50、YOLOv5、YOLOv6、YOLOv8等；
- **LLM模型：** Qwen 2.5 0.5B、1.5B、3B等；
- **VLM模型：** FastVLM、Qwen2.5 VL、Qwen3 VL等。

开发板运行环境： 模型转换完成后，可在开发板上使用RKNN3 API加载和运行RKNN模型，也可通过OpenAI兼容API调用LLM模型。主要运行组件如下：

组件	作用
RKNN3 API	提供RKNN模型加载、推理、LLM模型推理及会话管理等核心功能
rkllm3-server	提供OpenAI兼容API服务，支持文本、图片和语音输入（暂不支持视频输入）
rknn3_transfer_proxy	协处理器模式下负责Host端与RK1820/RK1828之间的通信，支持PCIe、USB和以太网连接
rknn3-server	非协处理器模式下运行在RK3572侧，负责处理来自PC端的连板推理请求

快速入门路径如下：

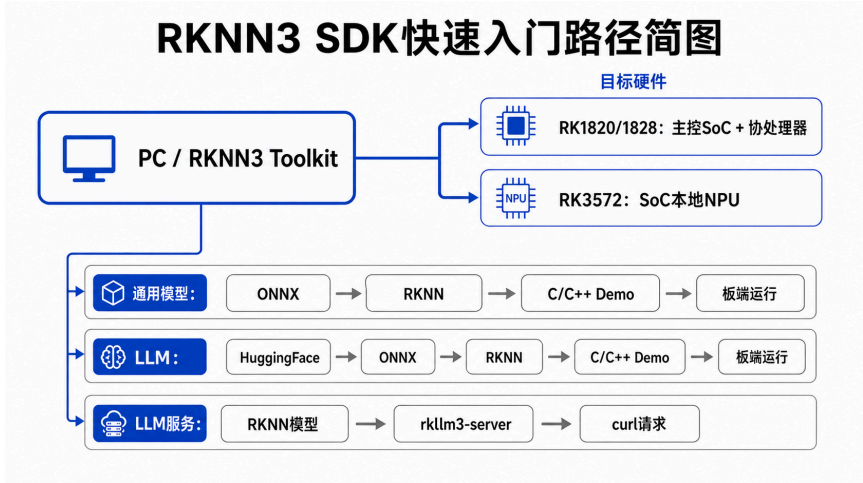


图1-3 RKNN3 SDK快速入门路径简图

软件栈各组件的完整关系、Runtime职责以及API分层请参考《RKNN3_SDK_开发指南》等文档。

1.3 选择示例类型

本指南使用两个示例贯穿首次运行流程：

示例	用途	Model Zoo目录
YOLOv6	第一个通用模型	examples/yolov6
Qwen2.5 3B	第一个LLM	examples/Qwen2_5

如果当前目标只是确认通用模型部署链路，完成第4章即可；如果需要部署LLM，再继续第5章。RKLLM3 Server为独立服务入口，可在完成LLM模型准备后继续第6章。

1.4 支持平台和版本要求

支持的硬件平台如下：

部署方式	硬件平台
协处理器相关平台	RK3588/RK3576 + RK1820/RK1828协处理器
协处理器相关平台	Windows PC + RK1820/RK1828协处理器
非协处理器平台	RK3572

支持的操作系统如下：

部署模式	操作系统
协处理器模式（RK182X）	Android、Linux、Windows
非协处理器模式（RK3572）	Android、Linux（仅aarch64）

开发主机示例基于Ubuntu 20.04/Ubuntu 22.04。RKNN3 Toolkit推荐使用Python 3.10环境；当前SDK同时提供Python 3.12相关安装包时，可按发布包实际内容选择。

注意：硬件平台、主控芯片、操作系统、Python版本以及组件版本的完整支持关系不在本指南重复维护，请统一参考SDK发布包中的共享兼容矩阵。

1.5 获取SDK及配套仓库

建议创建专门目录存放RKNN3仓库，例如创建名为 `Projects` 的文件夹，并将RKNN3相关仓库下载到该目录下。下载方式有SDK和Github两种方式，以下重点介绍Github下载方式。

注意： SDK离线发布包与GitHub仓库的目录结构可能不同，后续路径应以实际获取方式为准。

使用GitHub方式时，需要下载 `rknn3-toolkit` 和 `rknn3-model-zoo` 两个仓库，参考命令如下：

```
# 新建 Projects 文件夹
mkdir -p ~/Projects

# 进入该目录
cd ~/Projects

# 下载 rknn3-toolkit 仓库
git clone --depth 1 https://github.com/airockchip/rknn3-toolkit

# 下载 rknn3-model-zoo 仓库
git clone --depth 1 https://github.com/airockchip/rknn3-model-zoo
```

后续示例默认目录结构如下：

```
Projects
├── rknn3-model-zoo
│   ├── 3rdparty
│   └── examples
├── rknn3-toolkit
│   ├── doc
│   ├── rknn3-toolkit
│   ├── rknn3-toolkit-lite
│   └── rknn3-runtime
│       ├── rknn3-api
│       ├── rknn3_transfer_proxy
│       ├── rknn3-server
│       ├── rkllm3-server
│       └── ...
```

如果使用SDK离线发布包，请以发布包实际目录为准，不需要重复下载GitHub仓库。

2. 硬件连接与检查

本章只完成首次运行所需的硬件连接和设备可见性检查。硬件原理、完整连接拓扑及更多连接方式请参考《RKNN3_SDK_开发指南》等文档。

2.1 协处理器模式（RK182X）

2.1.1 硬件清单

运行本文档示例程序需要准备以下硬件：

硬件	数量
计算机	1
RK1820/RK1828模组	1
RK_EVB10_RK3588_V10开发板	1
USB-C数据线	1
RK3588电源适配器	1

说明： 以下章节中RK1820/RK1828模组简称RK1820/RK1828，RK_EVB10_RK3588_V10开发板简称RK3588。

2.1.2 开发板和连接工具介绍

RK1820/RK1828模组



图2-1 RK1820/RK1828模组

RK_EVB10_RK3588_V10开发板



图2-2 RK_EVB10_RK3588_V10开发板

USB-C数据线



图2-3 USB-C 数据线

电源适配器



图2-4 RK3588电源适配器

2.1.3 连接开发板

下面以RK_EVB10_RK3588_V10开发板搭载RK1820/RK1828为例说明如何快速开发：

1. 准备一台操作系统为 `Ubuntu20.04/Ubuntu22.04` 的计算机。

2. 将RK1820/RK1828模组插入到RK_EVB10_RK3588_V10开发板上，如下图所示：



图2-5 RK_EVB10_RK3588_V10开发板搭载RK1820/RK1828

1. 将RK1820/RK1828/RK3588开发板上标识为串口的端口通过数据线与计算机相连。（注意，由于硬件版本不同，开发板的数据线接口类型和位置可能会发生变化。）
2. 打开电源开关，等待开发板系统启动完成。
3. 查看开发板是否连接至计算机

1) 检查RK3588是否连接成功

在计算机的终端窗口中，执行以下命令：

```
# 安装 adb
sudo apt install adb

# 查询adb连接的设备
adb devices
```

连接成功时输出信息如下，其中 13af7b28115662cd 为RK3588的设备ID（示例值，实际以本机为准）。若无设备显示请参考[7.1 设备无法识别](#)。

```
List of devices attached
13af7b28115662cd    device
```

2) 检查RK1820/RK1828是否连接成功

在计算机的终端窗口中，执行以下命令：

```
# 进入RK3588终端
adb shell

rknn3_transfer_proxy devices
```

正常情况下应看到RK1820/RK1828设备，例如：

```
List of ntb devices attached
0000:01:00.0      b98e6c51      PCIE
```

注意：若找不到rknn3_transfer_proxy，请参考[3.3 安装或检查板端RKNPU3环境](#)。

如果 adb devices 无设备，参考[7.1 设备无法识别](#)；如果主控可见但协处理器不可见，也先参考[7.1 设备无法识别](#)中的协处理器检查项。

2.2 非协处理器模式（RK3572）

2.2.1 硬件清单

运行本文档示例程序需要准备以下硬件：

硬件	数量
计算机	1
RK3572开发板	1
USB-C数据线	1
RK3572电源适配器	1

说明： 以下章节中RK3572开发板简称RK3572。

2.2.2 开发板和连接工具介绍

RK3572开发板



图2-6 RK3572开发板

USB-C数据线



图2-7 USB-C 数据线

电源适配器



图2-8 RK3572电源适配器

2.2.3 连接开发板

1. 准备一台操作系统为 `Ubuntu20.04/Ubuntu22.04` 的计算机。
2. 将开发板上标识为串口的端口通过数据线与计算机相连。（注意，由于硬件版本不同，开发板的数据线接口类型和位置可能会发生变化。）
3. 打开电源开关，等待开发板系统启动完成。
4. 查看开发板是否连接至计算机

在计算机的终端窗口中，执行以下命令：

```
# 安装 adb
sudo apt install adb

# 查询adb连接的设备
adb devices
```

正常情况下应看到RK3572设备，例如：

```
List of devices attached
2f4a8c91e0d3b5a7    device
```

其中 2f4a8c91e0d3b5a7 为RK3572的设备ID（示例值，实际以本机为准）。若无设备显示请参考[7.1 设备无法识别](#)。

2.3 检查设备连接状态

在继续安装和模型转换前，至少确认以下检查全部通过：

检查项	适用模式	检查命令
ADB工具可用	两种模式	<code>adb version</code>
板端设备可见	两种模式	<code>adb devices</code>
RK1820/RK1828协处理器可见	协处理器模式	Android: <code>adb shell "/vendor/bin/rknn3_transfer_proxy devices"</code>
RK1820/RK1828协处理器可见	协处理器模式	Linux: <code>adb shell "/usr/bin/rknn3_transfer_proxy devices"</code>

上述检查均通过后，再继续主机端和板端环境准备。

3. 主机端和板端环境准备

3.1 安装RKNN3 Toolkit

快速入门仅给出推荐的Conda + 本地wheel安装路径。Docker、系统Python、多Python版本管理以及完整依赖说明请参考《RKNN3_SDK_开发指南》等文档。

3.1.1 安装Python环境

如果系统中未安装**Python 3.10（推荐版本）**或**Python 3.12**，或存在多个Python版本，建议使用Miniforge Conda创建独立的Python 3.10环境。

步骤1：安装Miniforge Conda

首先检查系统是否已安装Miniforge Conda；若已安装，可跳过本步骤。

```
conda -V
# 输出示例：conda 23.3.1，表示Miniforge conda版本为23.3.1
# 如果提示 conda: command not found，则表示未安装Miniforge
```

如果未安装Miniforge Conda，可通过以下命令下载安装包：

```
wget -c https://github.com/conda-forge/miniforge/releases/download/25.3.0-1/Miniforge3-25.3.0-1-Linux-x86_64.sh
```

然后执行以下命令安装Miniforge Conda：

```
# 拷贝到用户目录
cp /path/to/Miniforge3-25.3.0-1-Linux-x86_64.sh ~

# 增加运行权限
chmod 777 Miniforge3-25.3.0-1-Linux-x86_64.sh

# 运行安装脚本
bash Miniforge3-25.3.0-1-Linux-x86_64.sh
```

步骤2：创建Python环境

安装Miniforge Conda后，刷新环境变量并进入Conda base环境：

```
# 刷新环境变量
source ~/.bashrc

# 成功后，命令行提示符会变为：
# (base) xxx@xxx:~$
```

执行以下命令创建名为toolkit3的Python 3.10环境：

```
conda create -n toolkit3 python=3.10
```

激活toolkit3环境，后续将在此环境中安装rknn3-toolkit：

```
conda activate toolkit3
# 成功后，命令行提示符会变为：
# (toolkit3) xxx@xxx:~$
```

说明： 后续Python运行命令默认在 `toolkit3` 环境中执行。

3.1.2 安装rknn3-toolkit

激活toolkit3环境后，通过本地wheel包安装rknn3-toolkit：

```
cd ~/Projects/rknn3-toolkit/rknn3-toolkit

# 根据Python版本和处理器架构选择对应的requirements文件
# cpxxx为Python版本号，x.x.x为rknn3-toolkit版本号
pip install -r packages/requirements_cpxxx-x.x.x.txt

# 安装rknn3-toolkit
# 根据Python版本和处理器架构选择对应的wheel安装包
# x.x.x为rknn3-toolkit版本号，cpxx为Python版本号
pip install packages/x86_64/rknn3_toolkit-x.x.x-cpxx-cpxx-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

其中 `x.x.x`、`cpxx/cpxxx` 均需替换为当前SDK发布包中的实际版本。

3.2 检查Toolkit安装结果

保持 `toolkit3` 环境处于激活状态，执行：

```
python -c "from rknn.api import RKNN; print('RKNN3 Toolkit import OK')"
```

若输出：

```
RKNN3 Toolkit import OK
```

表示最基本的Toolkit导入检查通过。

3.3 安装或检查板端RKNPU3环境

板端运行C/C++ Demo前，需要准备Runtime库和对应的通信服务。各组件作用如下：

组件	适用模式	说明
librknn3_api.so	两种模式	负责加载RKNN模型，并通过相应接口调用NPU执行推理
rknn3_transfer_proxy	协处理器模式	运行在RK3588开发板上，通过PCIe/USB/Ethernet在RK3588与RK1820/RK1828之间传输数据
rknn3-server	非协处理器模式	运行在RK3572侧，负责处理来自PC端的连板推理请求

3.3.1 安装Runtime库和通信服务

首先进入 rknn3-runtime 目录：

```
cd ~/Projects/rknn3-toolkit/rknn3-runtime
```

1. 安装Runtime库

Android系统：

```
adb push rknn3-api/Android/arm64-v8a/librknn3_api.so /vendor/lib64/
adb push rknn3-api/Android/arm64-v8a/librknn3_api_rkcp.so /vendor/lib64/
adb push rknn3-api/Android/arm64-v8a/librknn3_api_native.so /vendor/lib64/
```

Linux系统：

```
adb push rknn3-api/Linux/aarch64/librknn3_api.so /usr/lib/
adb push rknn3-api/Linux/aarch64/librknn3_api_rkcp.so /usr/lib/
adb push rknn3-api/Linux/aarch64/librknn3_api_native.so /usr/lib/
```

2. 安装通信服务

协处理器模式（RK182X），Android系统：

```
adb push rknn3_transfer_proxy/android-arm64-v8a/rknn3_transfer_proxy /vendor/bin/
adb shell chmod +x /vendor/bin/rknn3_transfer_proxy
```

协处理器模式（RK182X），Linux系统：

```
adb push rknn3_transfer_proxy/linux-aarch64/rknn3_transfer_proxy /usr/bin/
adb shell chmod +x /usr/bin/rknn3_transfer_proxy
```

非协处理器模式（RK3572），Android系统：

```
adb push rknn3-server/Android/arm64-v8a/rknn3-server /vendor/bin/
adb shell chmod +x /vendor/bin/rknn3-server
```

非协处理器模式（RK3572），Linux系统：

```
adb push rknn3-server/Linux/aarch64/rknn3-server /usr/bin/  
adb shell chmod +x /usr/bin/rknn3-server
```

安装完成后执行：

```
adb shell sync
```

重要提示：

1. 将文件推送到开发板后，务必执行 `sync` 操作（PC端执行 `adb shell sync`，或进入板端后执行 `sync`）。
2. `rknn3_transfer_proxy` 在RK3588上可设置为开机自动启动；如未设置，需手动启动。

3.3.2 检查版本

通信服务与Runtime库应来自同一套SDK版本。如果组件缺失或版本不一致，请按照上述3.3.1步骤完成安装或更新。

1. 检查Runtime库版本

Android系统：

```
adb shell 'strings /vendor/lib64/librknn3_api.so | grep -i "librknn3_api version"'
```

Linux系统：

```
adb shell 'strings /usr/lib/librknn3_api.so | grep -i "librknn3_api version"'
```

2. 检查通信服务

RK182X：

```
# Android  
adb shell '/vendor/bin/rknn3_transfer_proxy'  
  
# Linux  
adb shell '/usr/bin/rknn3_transfer_proxy'
```

RK3572：

```
# Android  
adb shell '/vendor/bin/rknn3-server -v'  
  
# Linux  
adb shell '/usr/bin/rknn3-server -v'
```

完整更新步骤请参考《RKNN3_SDK_开发指南》等文档。

3.4 安装编译工具

3.4.1 安装CMake

```
# 更新包列表
sudo apt update

# 安装 cmake
sudo apt install cmake
```

3.4.2 安装编译器

3.4.2.1 确认开发板系统类型和架构

为便于描述，后续文档使用"板端"表示开发板端。

步骤1：确认板端系统类型

在终端中执行以下命令：

```
adb shell getprop ro.build.version.release
```

如果输出为数字（Android系统版本号），则表示板端为Android系统：

```
adb shell getprop ro.build.version.release

# 输出
12
```

否则板端为Linux系统：

```
adb shell getprop ro.build.version.release

# 输出
/bin/bash: line 1: getprop: command not found
```

步骤2：确认板端系统架构

如果板端为Android系统，可在PC端执行以下命令查询系统架构：

```
adb shell getprop ro.product.cpu.abi
```

该命令输出示例，其中arm64-v8a表示ARM 64位架构、第八版本ABI。

```
adb shell getprop ro.product.cpu.abi

# 输出
arm64-v8a
```

如果板端为Linux系统，可在PC端执行以下命令查询系统架构：

```
adb shell uname -a
```

该命令输出示例，其中aarch64表示ARM 64位架构。

```
adb shell uname -a

# 输出
Linux rk3588-buildroot 6.1.118 #4 SMP Sat Jul 5 14:44:53 CST 2025 aarch64
GNU/Linux
```

3.4.2.2 Android系统开发板安装NDK

适用范围： 本节适用于Android系统开发板。如果板端为Linux系统，请跳过本节。

- **NDK下载地址**（建议下载r19c版本）：https://dl.google.com/android/repository/android-ndk-r19c-linux-x86_64.zip
- **解压软件包**

建议将NDK软件包解压到Projects文件夹中，位置如下：

```
Projects
├── rknn3-toolkit
├── rknn3-model-zoo
└── android-ndk-r19c # 此路径在后续编译RKNN3 C Demo时会用到
```

此时NDK编译器路径为 Projects/android-ndk-r19c。

3.4.2.3 Linux系统开发板安装GCC交叉编译器

适用范围： 本节适用于Linux系统开发板。如果板端为Android系统，请跳过本节。

应用程序在开发板上运行，根据开发板SoC选择交叉编译工具。以Linux aarch64系统为例：

- **交叉编译工具下载地址**

```
https://dn.odroid.com/compiler/gcc-linaro-6.3.1-2017.05-x86\_64\_aarch64-linux-gnu.tar
```

- **解压软件包**

建议将GCC软件包解压到Projects文件夹中。以板端为64位系统的GCC软件包为例，存放位置如下：


```
Projects
├── rknn3-toolkit
├── rknn3-model-zoo
└── gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu # 此路径在后续编译RKNN3 C Demo
    时会用到
```

此时GCC编译器路径为 `Projects/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu`。

3.5 运行环境预检查

在开始模型转换前，建议一次完成以下检查：

```
# Toolkit
conda activate toolkit3
python -c "from rknn.api import RKNN; print('Toolkit OK!)"

# Model Zoo
cd ~/Projects/rknn3-model-zoo
ls examples/yolov6 examples/Qwen2_5

# 编译工具
cmake --version

# 设备
adb devices
```

如果需要运行LLM示例，还应安装Model Zoo的Python依赖：

```
cd ~/Projects/rknn3-model-zoo
pip install -r requirements.txt
```

4. 运行第一个通用模型

本章使用YOLOv6完成一次“获取模型 -> 转换 -> 编译 -> 板端运行 -> 检查结果”的最小闭环。完整通用模型开发流程、模型评估、量化和API调用顺序请参考《RKNN3_SDK_开发指南》与《RKNN3_Toolkit_Python_API_参考》文档。

4.1 获取示例

进入YOLOv6示例并下载模型：

```
cd ~/Projects/rknn3-model-zoo/examples/yolov6/model
bash download_model.sh
```

download_model.sh 中包含带 _rknn3 后缀的优化模型和不带该后缀的标准模型下载命令，请根据目标平台选择模型。相关输出：

```
--2025-09-15 11:30:44--
https://ftrg.zbox.filez.com/v2/delivery/data/95f00b0fc900458ba134f8b180b3f7a1/examples/yolov6/yolov6n_rknn3.onnx
Resolving ftrg.zbox.filez.com (ftrg.zbox.filez.com)... 180.184.171.46
Connecting to ftrg.zbox.filez.com (ftrg.zbox.filez.com)|180.184.171.46|:443...
connected.
HTTP request sent, awaiting response... 200
Length: 18644871 (18M) [application/octet-stream]
Saving to: './yolov6n_rknn3.onnx'

./yolov6n_rknn3.onnx          100%[=====>]
17.78M  2.48MB/s   in 7.9s

2025-09-15 11:30:53 (2.26 MB/s) - './yolov6n_rknn3.onnx' saved [18644871/18644871]
```

注意：

1. 文件名带 _rknn3 后缀的模型将YOLO后处理（解码、候选框筛选排序以及NMS等）内置到模型中计算；不带 _rknn3 后缀的标准模型由应用程序完成后处理。RKNN3 Model Zoo中的 yolov6n_rknn3.onnx 面向RK182X优化，RK3572示例请使用标准模型 yolov6n.onnx。
2. 从以下链接下载完整的PyTorch示例工程：[pytorch示例](#)。下载完成后，参考其中的导出文档，即可生成适配RKNN3的优化ONNX模型。下载链接也可在 download_model.sh 中查看。

4.2 转换模型

激活Toolkit环境并进入转换脚本目录：

```
conda activate toolkit3
cd ~/Projects/rknn3-model-zoo/examples/yolov6/python
```

按目标平台执行对应的转换命令。

RK182X:

```
python convert.py ../model/yolov6n_rknn3.onnx rk1820 i8
```

RK3572:

```
python convert.py ../model/yolov6n.onnx rk3572 i8
```

命令格式为 `python convert.py onnx_model_path platform [dtype] [output_rknn_path]`。

其中 `i8` 表示启用8位量化；`dtype` 省略时脚本默认启用8位量化；`output_rknn_path` 省略时在输入模型目录生成同名 `.rknn` 和 `.weight` 文件。

注意： `convert.py` 脚本的帮助提示中仅列出了 `rk1820`，但 `platform` 参数会直接透传给RKNN3 Toolkit，因此 `rk3572` 同样适用。

转换完成后，模型目录中应出现对应的 `.rknn` 和 `.weight` 文件。相关输出：

```

register configuration: [=====]
1/1 (100.0%) Total: 0s
memory optimization: [=====] 1/1
(100.0%) Total: 0s
weight sharing: [=====] 1/1
(100.0%) Total: 0s
code generation: [=====] 1/1
(100.0%) Total: 1s
I rknn building done.
done
--> Export rknn model
done

```

4.3 编译部署程序

4.3.1 Android

Android系统使用 `build-android.sh` 编译，目标架构为arm64-v8a。

1. 配置NDK路径

在 `build-android.sh` 脚本开头设置 `ANDROID_NDK_PATH`：

```
ANDROID_NDK_PATH=Projects/android-ndk-r19c
```

2. 编译Demo

```

cd ~/Projects/rknn3-model-zoo

# RK182X: 应用运行在RK3588/RK3576等主控SoC
./build-android.sh -t rk3588 -a arm64-v8a -b Release -d yolov6

# RK3572
./build-android.sh -t rk3572 -a arm64-v8a -b Release -d yolov6

```

3. 部署Demo

```

# RK182X
adb push install/rk3588_android_arm64-v8a/rknn_yolov6_demo /data/local/tmp/

# RK3572
adb push install/rk3572_android_arm64-v8a/rknn_yolov6_demo /data/local/tmp/

```

Android调试阶段推荐使用 `/data/local/tmp`，避免 `/data` 目录权限不足。若固件允许使用 `/data`，也可按项目实际部署目录调整。

4.3.2 Linux

Linux系统使用 `build-linux.sh` 编译，目标架构为aarch64。

1. 配置交叉编译器

```
export GCC_COMPILER=~/.opt/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-  
gnu/bin/aarch64-linux-gnu  
cd ~/Projects/rknn3-model-zoo
```

2. 编译Demo

```
# RK182X: 应用运行在RK3588/RK3576等主控SoC  
./build-linux.sh -t rk3588 -a aarch64 -b Release -d yolov6  
  
# RK3572  
./build-linux.sh -t rk3572 -a aarch64 -b Release -d yolov6
```

3. 部署Demo

```
# RK182X  
adb push install/rk3588_linux_aarch64/rknn_yolov6_demo /data/  
  
# RK3572  
adb push install/rk3572_linux_aarch64/rknn_yolov6_demo /data/
```

4.4 板端运行

首先进入板端：

```
adb shell
```

1. 进入Demo目录并配置动态库路径

Android：

```
cd /data/local/tmp/rknn_yolov6_demo  
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

Linux：

```
cd /data/rknn_yolov6_demo  
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

2. 运行Demo

命令格式：

```
./rknn_yolov6_demo <model_path> <weight_path> <image_path> <core_mask>
```

RK182X：

```
./rknn_yolov6_demo \  
./model/yolov6n_rknn3.rknn \  
./model/yolov6n_rknn3.weight \  
./model/bus.jpg 1
```

RK3572:

```
./rknn_yolov6_demo \  
./model/yolov6n.rknn \  
./model/yolov6n.weight \  
./model/bus.jpg 1
```

4.5 验证输出结果

成功运行时，终端应打印推理时间、目标类别、置信度和目标框，例如：

```
model is NHWC input fmt  
model input height=640, width=640, channel=3  
origin size=640x640 crop size=640x640  
input image: 640 x 640, subsampling: 4:2:0, colorspace: YCbCr, orientation: 1  
--> inference model  
scale=1.000000 dst_box=(0 0 639 639) allow_slight_change=1 _left_offset=0  
_top_offset=0 padding_w=0 padding_h=0  
rga_api version 1.10.1_[0]  
Pre-process time: 2.09 ms  
-->rknn_run  
Inference time: 21.37 ms  
Post-process time: 0.44 ms  
Total time: 23.90 ms  
--> inference model done  
bus @ (97 140 557 440) 0.951  
person @ (109 239 221 537) 0.940  
person @ (213 238 287 511) 0.932  
person @ (480 233 560 520) 0.924  
person @ (78 326 117 515) 0.455  
write_image path: out.png width=640 height=640 channel=3 data=0x7f885e801
```

如需查看结果图片，可在开发主机执行：

```
# Android示例路径  
adb pull /data/local/tmp/rknn_yolov6_demo/out.png .  
  
# Linux示例路径  
adb pull /data/rknn_yolov6_demo/out.png .
```

实际检测结果会随模型版本、输入图片及运行环境变化；首次运行只需确认Demo正常结束并生成 `out.png`。

5. 运行第一个LLM

本章使用Qwen2.5 3B完成一次LLM模型导出、RKNN转换、C/C++ Demo编译和板端推理。LLM的ONNX约束、KVCache、Session、LoRA、量化配置等完整说明请参考《RKNN3_SDK_开发指南》及《RKNN3_Runtime_C_API_参考》。

5.1 获取示例

确认Model Zoo中存在Qwen2.5示例：

```
cd ~/Projects/rknn3-model-zoo
ls examples/Qwen2_5
```

安装示例依赖：

```
conda activate toolkit3
pip install -r requirements.txt
```

如果示例需要从HuggingFace下载原始模型，请按 `examples/Qwen2_5` 目录中的README或脚本默认配置准备模型路径，本指南不重复维护各模型下载方式。

5.2 转换模型

LLM首次运行主要包括模型导出、RKNN转换和板端运行三个阶段。本节先完成模型导出与转换。

首先设置Model Zoo Python路径：

```
cd ~/Projects/rknn3-model-zoo
export PYTHONPATH="$PWD:$PYTHONPATH"
cd examples/Qwen2_5/python
```

步骤1：导出ONNX及LLM配套文件

```
python export_llm.py
```

运行成功后会在 `examples/Qwen2_5/model` 文件夹下生成后缀名为 `onnx` 的模型文件、后缀名为 `.config.pkl` 的配置文件，以及tokenizer和embedding等文件。

步骤2：转换为RKNN模型

RK182X：

```
python export_rknn.py --platform rk1820
```

RK3572:

```
python export_rknn.py --platform rk3572
```

相关输出:

```
I rknn building done.  
done  
--> Export RKNN model  
done
```

完成上述步骤后应得到对应的 `.rknn` 和 `.weight` 模型文件，以及板端运行所需的 `.tokenizer.gguf`、`.embed.bin` 等文件。

5.3 编译部署程序

5.3.1 Android

Android系统使用 `build-android.sh` 编译，目标架构为arm64-v8a。

1. 配置NDK路径

```
ANDROID_NDK_PATH=Projects/android-ndk-r19c
```

2. 编译Demo

```
cd ~/Projects/rknn3-model-zoo  
  
# RK182X  
./build-android.sh -t rk3588 -a arm64-v8a -b Release -d Qwen2_5  
  
# RK3572  
./build-android.sh -t rk3572 -a arm64-v8a -b Release -d Qwen2_5
```

3. 部署Demo

```
# RK182X  
adb push install/rk3588_android_arm64-v8a/rknn_Qwen2_5_demo /data/local/tmp/  
  
# RK3572  
adb push install/rk3572_android_arm64-v8a/rknn_Qwen2_5_demo /data/local/tmp/
```

5.3.2 Linux

Linux系统使用 `build-linux.sh` 编译，目标架构为aarch64。

1. 配置交叉编译器


```
export GCC_COMPILER=~/.opt/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu/bin/aarch64-linux-gnu
cd ~/Projects/rknn3-model-zoo
```

2. 编译Demo

```
# RK182X
./build-linux.sh -t rk3588 -a aarch64 -b Release -d Qwen2_5

# RK3572
./build-linux.sh -t rk3572 -a aarch64 -b Release -d Qwen2_5
```

3. 部署Demo

```
# RK182X
adb push install/rk3588_linux_aarch64/rknn_Qwen2_5_demo /data/

# RK3572
adb push install/rk3572_linux_aarch64/rknn_Qwen2_5_demo /data/
```

5.4 板端运行

首先进入板端：

```
adb shell
```

1. 进入Demo目录并配置动态库路径

Android：

```
cd /data/local/tmp/rknn_Qwen2_5_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

Linux：

```
cd /data/rknn_Qwen2_5_demo
export LD_LIBRARY_PATH=./lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}
```

2. 运行Qwen2.5 Demo

命令格式：

```
./rknn_qwen2_5_demo <model_path> <weight_path> <tokenizer_path> <embedding_path>
<core_mask> <prompt>
```

RK182X：

```
./rknn_qwen2_5_demo \  
./model/Qwen2.5-3B-Instruct.rknn \  
./model/Qwen2.5-3B-Instruct.weight \  
./model/Qwen2.5-3B-Instruct.tokenizer.gguf \  
./model/Qwen2.5-3B-Instruct.embed.bin \  
0xff \  
"who are you?"
```

RK3572:

```
./rknn_qwen2_5_demo \  
./model/Qwen2.5-3B-Instruct.rknn \  
./model/Qwen2.5-3B-Instruct.weight \  
./model/Qwen2.5-3B-Instruct.tokenizer.gguf \  
./model/Qwen2.5-3B-Instruct.embed.bin \  
0x01 \  
"who are you?"
```

如果Android系统执行 `adb push ... /data` 时提示 `Permission denied`，调试阶段可将Demo推送到 `/data/local/tmp`，并将后续 `cd` 路径同步改为该目录。

5.5 验证输出结果

首次运行主要确认以下三点：

1. 模型、权重、tokenizer和embedding文件均能正常加载；
2. Runtime初始化和Session推理过程中没有报错；
3. 终端能够输出模型生成的文本。

不同模型版本和采样配置会产生不同回复，因此不要求输出文本与本文完全一致。

6. 启动RKLLM3 Server

`rkllm3-server` 是基于RKNPU3实现的一套基本的LLM Server，提供OpenAI API兼容的对话接口。本章演示如何启动服务、发送请求并验证结果。

主要功能如下：

功能	说明
LLM推理	基于RKNPU3执行LLM推理
OpenAI兼容接口	提供OpenAI API兼容的对话接口
多模态输入	支持文本、图片和语音输入，暂不支持视频输入

说明： 本章仅覆盖“启动服务并完成一次请求”所需的最小步骤。多模型加载、多Session、Slot暂停/恢复/打断、KV Cache保存/加载、检查点、LoRA、Tie Word Embedding、VLM/Audio/Video输入、完整API端点定义和错误码等高级功能，请参考RKLLM3_Server_使用指南及Server API参考/OpenAPI文档。

6.1 常用参数

启动 `rkllm3-server` 时需要指定模型文件、词汇表、embedding等参数。以下是快速入门所需的最常用参数：

通用参数

参数	说明
<code>-m, --model FNAME</code>	LLM模型路径
<code>--weight FNAME</code>	LLM权重路径（可不设置）
<code>--vocab FNAME</code>	词汇表路径
<code>--embed FNAME</code>	embed的bin文件路径
<code>-c, --ctx-size N</code>	提示词上下文大小（默认：4096，0 = 从模型加载）
<code>-n, --predict, --n_predict N</code>	要预测的token数量（默认：-1，-1 = 上下文大小）
<code>-a, --alias STRING</code>	设置模型别名（供REST API使用）
<code>--core-mask MASK</code>	LLM模型执行的NPU核心掩码（例如 <code>0xff</code> = 全部核心）。默认：自动

采样参数

参数	说明
<code>--temp N</code>	温度（默认：0.8）
<code>--top-k N</code>	top-k采样（默认：40，0 = 禁用）
<code>--top-p N</code>	top-p采样（默认：0.9，1.0 = 禁用）
<code>--repeat-penalty N</code>	惩罚重复token序列（默认：1.0，1.0 = 禁用）
<code>--presence-penalty N</code>	重复alpha存在惩罚（默认：0.0，0.0 = 禁用）
<code>--frequency-penalty N</code>	重复alpha频率惩罚（默认：0.0，0.0 = 禁用）

服务参数

参数	说明
<code>--host HOST</code>	监听IP地址（默认：127.0.0.1）
<code>--port PORT</code>	监听端口（默认：8080）
<code>-to, --timeout N</code>	服务器读写超时时间（秒）（默认：600）
<code>--device-id STRING</code>	设备ID（多设备时需要指定具体的设备ID）
<code>--log-level N</code>	日志等级（默认：0）
<code>-h, --help, --usage</code>	打印使用说明并退出

完整参数列表（包括多模态、LoRA、检查点、Tie Word Embedding等高级参数）请参考《RKLLM3_Server_使用指南》。

6.2 启动服务

启动服务前，从当前SDK的 `rknn3-runtime/rkllm3-server/bin` 目录选择与板端系统匹配的 `rkllm3-server` 可执行文件，并与Qwen2.5模型文件放到同一工作目录。

1. 准备运行文件

工作目录中至少包含以下文件：

```
rkllm3-server
Qwen2.5-3B-Instruct.rknn
Qwen2.5-3B-Instruct.weight
Qwen2.5-3B-Instruct.tokenizer.gguf
Qwen2.5-3B-Instruct.embed.bin
```

2. 启动服务

赋予可执行权限后，按模型配置启动服务：

```
./rkllm3-server \
-m Qwen2.5-3B-Instruct.rknn \
--weight Qwen2.5-3B-Instruct.weight \
--vocab Qwen2.5-3B-Instruct.tokenizer.gguf \
--embed Qwen2.5-3B-Instruct.embed.bin \
--host 0.0.0.0 \
--port 8080 \
-c 768 \
--n_predict 512 \
--repeat-penalty 1.1 \
--presence-penalty 1.0 \
--frequency-penalty 1.0 \
--top-k 1 \
--top-p 0.8 \
--temp 0.8
```

3. 检查启动状态

启动成功后，终端将显示类似以下信息：

```
load RKNN-LLM model: Qwen2.5-3B-Instruct.rknn
llm service is ready on http://0.0.0.0:8080
```

如果当前模型或SDK版本的文件名、上下文长度或启动参数不同，请以对应模型示例和《RKLLM3_Server_使用指南》为准。

6.3 健康检查

启动服务后，可通过健康检查接口确认服务是否就绪：

```
curl http://localhost:8080/health
```

健康检查结果如下：

HTTP状态	含义	典型响应
503	模型正在加载	<pre>{"error": {"code": 503, "message": "Loading model", "type": "unavailable_error"}}</pre>
200	模型已成功加载	<pre>{"status": "ok"}</pre>

6.4 使用curl发送请求

服务就绪后，使用OpenAI兼容的 /v1/chat/completions 端点发送对话请求：

```
curl --request POST \
  --url http://localhost:8080/v1/chat/completions \
  --header "Content-Type: application/json" \
  --header "Authorization: Bearer no-key" \
  --data '{
    "messages": [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": "Hello!"}
    ],
    "n_predict": 128
  }'
```

如果 `curl` 运行在另一台主机，请将 `localhost` 替换为运行 `rkllm3-server` 设备的IP地址。

6.5 使用Python OpenAI库发送请求

除了 `curl`，也可以使用Python的 `openai` 库发送请求，调用方式与标准OpenAI API一致：

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8080/v1", # 替换为实际服务地址
    api_key="sk-no-key-required"
)

completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Write a limerick about python exceptions"}
    ]
)

print(completion.choices[0].message.content)
```

如需流式输出，设置 `stream=True`：

```
completion = client.chat.completions.create(
    model="Qwen2.5-3B",
    messages=[{"role": "user", "content": "Hello!"}],
    stream=True,
    extra_body={"n_predict": 128}
)

for chunk in completion:
    delta = chunk.choices[0].delta
    if delta.content:
        print(delta.content, end='', flush=True)
    if chunk.choices[0].finish_reason == "stop":
        break
print('')
```

`extra_body` 支持的非OpenAI标准选项：

选项	说明
<code>n_predict</code>	当前请求要预测的token数量
<code>id_slot</code>	指定当前slot的id（多session时使用，默认为0）
<code>chat_template_kwargs</code>	对话模板参数，如 <code>{"enable_thinking": False}</code> 控制thinking输出
<code>save_kvcache_path</code> / <code>load_kvcache_path</code>	保存/加载KV Cache（两者不能同时使用）
<code>top_k</code> 、 <code>top_p</code> 、 <code>temperature</code> 等	采样参数，覆盖启动时的默认值

6.6 检查响应结果

请求成功时返回JSON响应，格式与OpenAI API一致，例如：

```
{
  "id": "chatcmpl-xxx",
  "object": "chat.completion",
  "model": "Qwen2.5-3B",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Hello! I am Qwen, a helpful AI assistant. How can I help you today?"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 20,
    "completion_tokens": 18,
    "total_tokens": 38
  }
}
```

首次验证只需确认：

- HTTP请求能够到达服务（返回HTTP 200）；
- 返回内容为有效JSON；
- 响应中 `choices[0].message.content` 包含模型生成结果。

7. 常见启动问题与下一步

本章收录首次运行过程中可能遇到的问题及排查方法。完整FAQ、模型转换问题、精度问题、性能问题、内存问题和高级部署问题请统一参考《RKNN3_SDK_开发指南》等文档的故障排查章节。

7.1 设备无法识别

如果 `adb devices` 没有设备，可以尝试以下方式来解决此问题：

1. 检查连线是否正确、重新插拔数据线、换计算机另一个USB端口来连接数据线、更换数据线。
2. 当使用USB连接开发板时，请确保本地计算机和Docker容器中同时只开启一个adb server服务。例如，如果需要在Docker容器中连接开发板，请在计算机的终端中执行命令 `adb kill-server` 终止本地计算机上的adb server服务。
3. 出现以下错误时，表示系统中未安装adb。需要执行安装命令 `sudo apt install adb` 安装adb。

```
command 'adb' not found, but can be installed with:
sudo apt install adb
```

7.2 设备/协处理器连接问题

由于协处理器模式采用主从式架构，主控SoC通过PCIe/USB/Ethernet接口连接协处理器，例如RK3588 ⇌ PCIe/USB/Ethernet ⇌ RK1820/RK1828。如果主控SoC可见但协处理器不可见，可以尝试以下方式来解决此问题：

1. 检查PCIe/USB/Ethernet连线是否正确、重新插拔设备、更换数据连接线。
2. 调用以下命令测试是否有正确输出：

```
./rknn3_transfer_proxy devices
# 参考输出如下
List of ntb devices attached
0000:01:00.0      b98e6c51      PCIE
```

1. 确认 `rknn3_transfer_proxy` 是否已正确安装和启动。若找不到 `rknn3_transfer_proxy`，请参考[3.3 安装或检查板端RKNPU3环境](#)。

7.3 Toolkit安装验证失败

先确认当前使用的是预期Conda环境：


```
conda activate toolkit3
python --version
which python
```

然后重新执行最小导入测试：

```
python -c "from rknn.api import RKNN; print('RKNN3 Toolkit import OK')"
```

如果仍失败，优先检查：

- Python版本是否与wheel文件的 cpXX 一致；
- requirements文件是否与当前Toolkit版本一致；
- wheel是否来自当前SDK发布包；
- 是否在错误的Python/Conda环境中执行安装。

完整安装与依赖问题请参考《RKNN3_SDK_开发指南》等文档。

7.4 转换模型时找不到'py_utils'

当出现 `ModuleNotFoundError: No module named 'py_utils'` 时，说明Python环境变量设置错误，需要正确设置环境变量。请根据实际rknn3-model-zoo目录路径进行设置。

例如，如果rknn3-model-zoo位于 `/home/rockchip/rknn3-model-zoo`，则设置为：

```
export PYTHONPATH="/home/rockchip/rknn3-model-zoo:$PYTHONPATH"
echo $PYTHONPATH # 确认环境变量设置正确
```

7.5 GRQ量化失败

首先检查环境中是否存在CUDA，外部GRQ量化仅支持在CUDA环境中运行。遇到不支持的模型时，可关闭外部GRQ量化，使用RKNN3 Toolkit工具进行normal量化。

7.6 板端Runtime版本不匹配

通信服务和Runtime库版本不一致时，可能导致初始化、连板推理或板端推理失败。请分别检查：

```
# Runtime: Android
adb shell 'strings /vendor/lib64/librknn3_api.so | grep -i "librknn3_api version"'

# Runtime: Linux
adb shell 'strings /usr/lib/librknn3_api.so | grep -i "librknn3_api version"'
```

RK182X检查：

```
# Android
adb shell '/vendor/bin/rknn3_transfer_proxy'

# Linux
adb shell '/usr/bin/rknn3_transfer_proxy'
```

RK3572检查：

```
# Android
adb shell '/vendor/bin/rknn3-server -v'

# Linux
adb shell '/usr/bin/rknn3-server -v'
```

如果版本不一致，请使用同一SDK发布包中的Runtime库和通信服务进行统一更新。完整更新步骤请参考《RKNN3_SDK_开发指南》等文档。

7.7 后续文档导航

完成首次运行后，请根据任务进入唯一主文档，避免在快速入门中继续查找完整API或高级配置。

需要继续了解的内容	主文档/入口
RKNN3软件栈、组件关系、通用模型/LLM/VLM完整开发流程	《RKNN3_SDK_开发指南》
硬件、OS、版本支持范围	《RKNN3_SDK_开发指南》
RKNN3 Toolkit完整安装与模型开发说明	《RKNN3_SDK_开发指南》
Toolkit方法、参数和返回值	《RKNN3_Toolkit_Python_API_参考》
RKNN3 C函数、结构体、枚举和错误码	《RKNN3_Runtime_C_API_参考》
RKNN3 Toolkit Lite Python方法和数据类型	《RKNN3_Toolkit_Lite_Python_API_参考》
RKLLM3 Server部署与高级能力	《RKLLM3_Server_使用指南》
RKLLM3 Server端点和请求/响应定义	《RKLLM3_Server_使用指南》
精度、性能、内存分析与系统化故障排查	《RKNN3_SDK_开发指南》等文档故障排查章节

说明：未在本指南中给出固定文件名的配套文档，请以当前SDK发布包中的实际文档名称为准。